

目录

- TOC {toc}

排序算法将列表中的数据元素按特定顺序排列。排序是现代数据和信息服务的基础，因为如果数据集按正确的顺序排列，从数据集中检索信息的计算复杂性可以显著降低。例如，排序常用于将数据规范化，以便快速比较和对数据列表进行调和。此外，许多数据处理算法的效率可以在数据处于特定顺序时得到提高。由于其重要性，高效的排序算法成为了许多计算机科学研究的课题。即使有了这些高效的算法，对大型数据列表进行排序仍然耗时，并且可以从并行执行中受益。并行化高效排序算法具有挑战性，需要精心设计。本章介绍了两种重要高效排序算法的并行设计：基数排序和归并排序。本章的大部分内容专注于基数排序；归并排序则基于第12章《归并》中介绍的并行归并模式进行了简要讨论。其他流行的并行排序算法，如转置排序和采样排序，也进行了简要讨论。

13.1 背景

排序是计算机最早的应用之一。排序算法将列表中的元素按一定顺序排列。排序算法所强制执行的顺序取决于这些元素的性质。常见的顺序包括数字的数值顺序和文本字符串的字典顺序。更正式地说，任何排序算法必须满足以下两个条件：

- 输出是非递减或非递增的顺序。对于非递减顺序，每个元素按照所需顺序都不小于前一个元素。对于非递增顺序，每个元素按照所需顺序都不大于前一个元素。
- 输出是输入的排列。即算法必须保留所有原始输入元素，同时将它们重新排序到输出中。

在其最简单的形式中，可以根据每个元素的值对列表进行排序。例如，列表 [5, 2, 7, 1, 3, 2, 8] 可以排序成非递减顺序的输出 [1, 2, 2, 3, 5, 7, 8]。

更复杂和常见的用例是每个元素由一个键字段和一个值字段组成，列表应根据键字段进行排序。例如，假设每个元素是一个元组（年龄，收入，单位为千美元）。列表 [(30,150), (32,80), (22,45), (29,80)] 可以按收入作为键字段排序成非递减顺序 [(30,150), (32,80), (29,80), (22,45)]。

排序算法可以分为稳定算法和不稳定算法。当两个元素的键值相等时，稳定排序算法会保留它们原来的出现顺序。例如，当使用收入作为键字段对列表 [(30,150), (32,80), (22,45), (29,80)] 进行非递减排序时，稳定排序算法必须保证 (32, 80) 出现在 (29,80) 之前，因为前者在原始输入中出现在后者之前。不稳定排序算法则不提供这种保证。如果希望使用多个键按级联方式排序列表，则需要使用稳定算法。例如，如果每个元素有一个主键和一个次键，使用稳定排序算法，可以先根据次键排序列表，然后再根据主键排序。第二次排序将保留第一次排序产生的顺序。

排序算法还可以分为基于比较和非比较的算法。基于比较的排序算法在排序 N 个元素的列表时，无法实现优于 $O(N \log N)$ 的复杂度，因为它们必须在元素之间执行最少数量的比较。相反，一些非比较的排序算法可以实现优于 $O(N \log N)$ 的复杂度，但它们可能无法推广到任意类型的键。基于比较和非比较的排序算法都可以并行化。在本章中，我们将介绍一种并行非比较的排序算法（基数排序）和一种并行基于比较的排序算法（归并排序）。

由于排序的重要性，计算机科学研究社区基于丰富多样的数据结构和算法策略，产生了大量的排序算法。因此，计算机科学入门课程经常使用排序算法来说明各种核心算法概念，如大 O 表示法；分治算法；堆和二叉树等数据结构；随机算法；最佳、最坏和平均情况分析；时间和空间的权衡；以及上限和下限。在本章中，我们继续这一传统，使用两种排序算法来说明几种重要的并行化和性能优化技术 (Satish et al., 2009)。

13.2 基数排序

一种非常适合并行化的排序算法是基数(Radix)排序。基数排序是一种非比较排序算法，通过基于一个基数值（或位置数制中的基数）将待排序的键分配到不同的桶中来工作。如果键包含多个数字，则对每个数字重复分配，直到所有数字都被处理完毕。每次迭代都是稳定的，保留了每个桶中前一次迭代的键的顺序。在处理表示为二进制数的键时，选择一个2的幂次作为基数是方便的，因为这使得迭代和提取数字变得容易。每次迭代本质上处理键中的一个固定大小的比特片。我们将从使用基数2（即1比特基数）开始，然后在本章稍后扩展到更大的基数值。

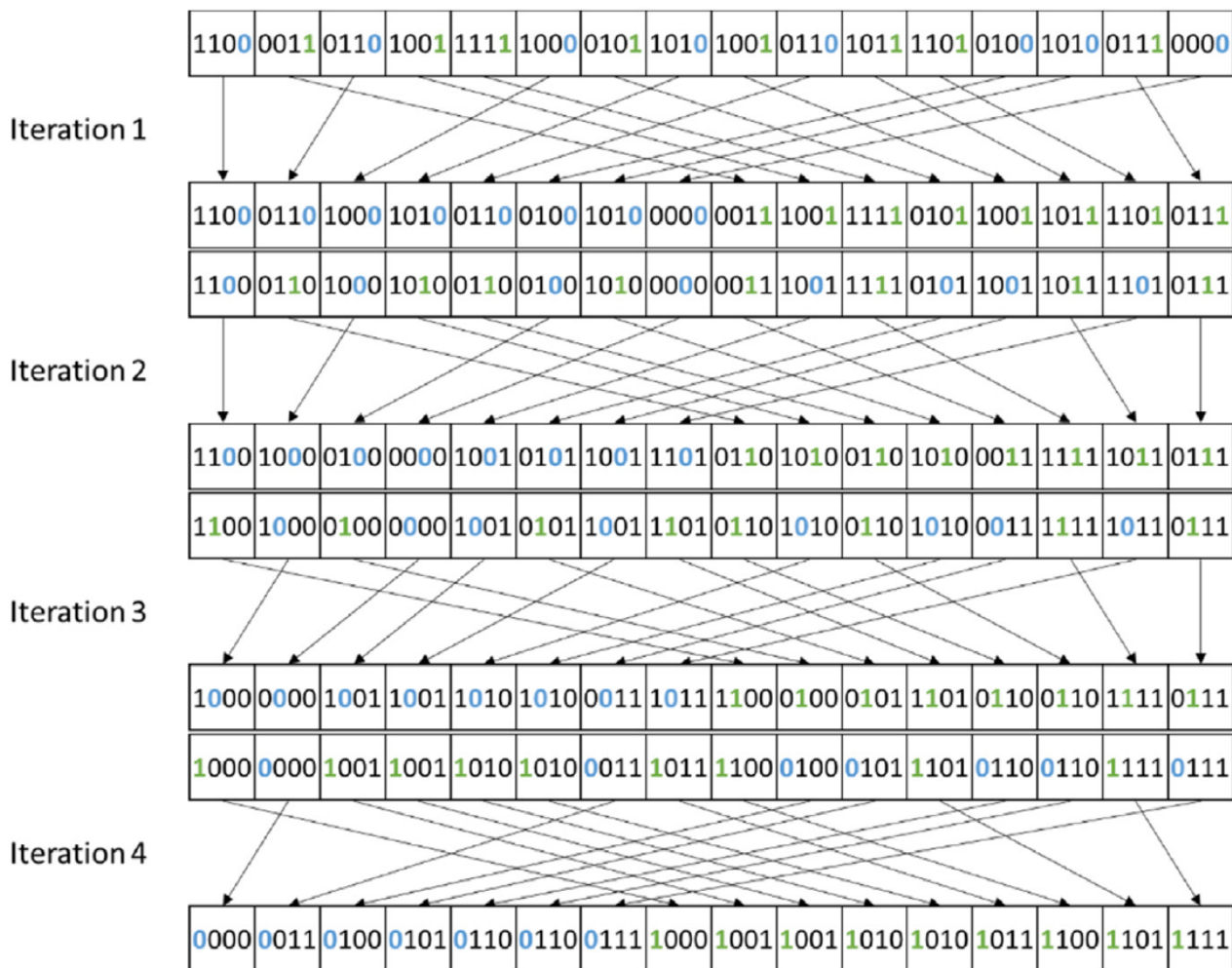


图13.1 基数排序示例

图13.1显示了如何使用1比特基数对4比特整数列表进行基数排序的示例。由于键是4比特长，每次迭代处理1比特，总共需要四次迭代。在第一次迭代中，考虑最不重要的比特（LSB）。所有在这次迭代的输入列表中LSB为0的键都放在这次迭代的输出列表的左侧，形成一个0比特的桶。同样，所有在这次迭代的输入列表中LSB为1的键都放在这次迭代的输出列表的右侧，形成一个1比特的桶。注意，在输出列表的每个桶中，键的顺序与输入列表中的顺序保持一致。换句话说，放在同一个桶中的键（即具有相同LSB的键）在输出列表中的出现顺序必须与它们在输入列表中的顺序相同。当我们讨论下一次迭代时，将会明白这种稳定性要求的重要性。

在图13.1的第二次迭代中，第一次迭代的输出列表成为新的输入列表，并考虑每个键的第二个LSB。与第一次迭代一样，键被分为两个桶：一个桶包含第二个LSB为0的键，另一个桶包含第二个LSB为1的键。由于保留了前一次迭代的顺序，我们观察到第二次迭代的输出列表中的键现在按低两位比特排序。换句话说，所有低两位比特为00的键首先出现，然后是低两位比特为01的键，接着是低两位比特为10的键，最后是低两位比特为11的键。

在图13.1的第三次迭代中，考虑键中的第三个比特，重复相同的过程。同样，由于保留了前几次迭代的顺序，第三次迭代的输出列表中的键按低三位比特排序。最后，在第四次也是最后一次迭代中，考虑第四个或最重要的比特，重复相同的过程。在这次迭代结束时，最终输出列表中的键按所有四个位比特排序。

13.3 并行基数排序

在基数排序中，每次迭代都依赖于前一次迭代的整个结果。因此，这些迭代必须依次顺序执行。并行化基数排序的机会在于每次迭代的内部。本章的剩余部分将集中讨论单次基数排序迭代的并行化，并假定这些迭代将依次执行。换句话说，我们将专注于实现一个执行单次基数排序迭代的内核，并假设主机代码在每次迭代时调用该内核一次。

在GPU上并行化基数排序的一种简单方法是让每个线程负责输入列表中的一个键。线程必须识别该键在输出列表中的位置，然后将该键存储到该位置。图13.2展示了应用于图13.1第一次迭代的这种并行化方法。图13.2中的线程用弯曲的箭头表示，线程块用箭头周围的框表示。每个线程负责输入列表中其下方的键。在这个例子中，16个键由一个包含四个线程块、每个线程块包含四个线程的网格处理。实际上，每个线程块可以有多达1024个线程，并且输入数据要大得多，从而产生更多的线程块。然而，我们使用较少的线程数来简化说明。

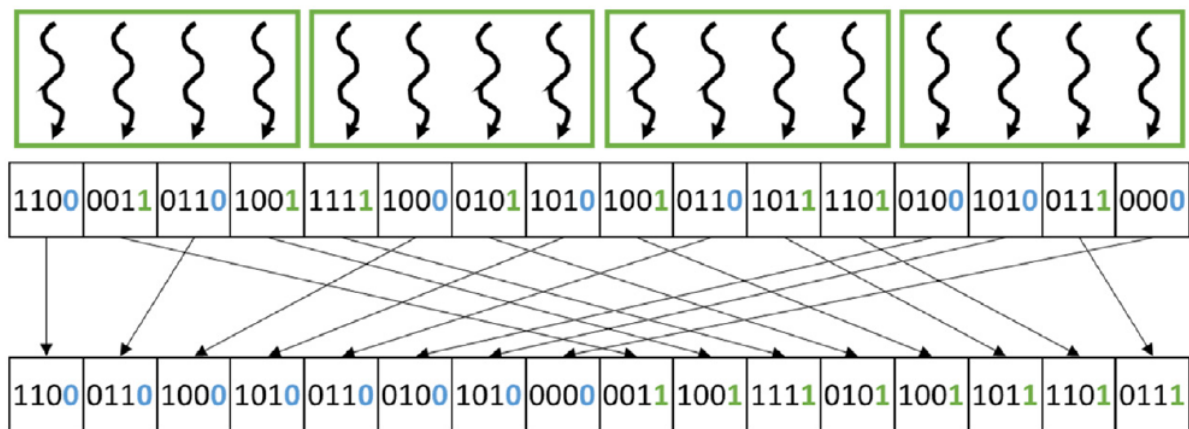


图13.2 通过将每个输入键分配给每个线程来并行化基数排序的迭代。

在每个线程分配到输入列表中的一个键之后，每个线程仍需要确定其键在输出列表中的目标索引。确定键的目标索引取决于该键映射到0桶还是1桶。对于映射到0桶的键，可以通过以下方法找到目标索引：

```
destination of a zero = # zeros before
                      = # keys before - # ones before
                      = key index - # ones before
```

映射到0桶的键（即0的目标）的目标索引等于该键之前映射到0桶的键的数量（即之前的0的数量）。由于所有键都映射到0桶或1桶，因此映射到0桶的键之前的键的数量等于该键之前的总键数（即之前的键的数量）减去映射到1桶的键的数量（即之前的1的数量）。该键之前的总键数只是键在输入列表中的索引（即键索引），这是显而易见的。因此，找到映射到0桶的键的目标索引的唯一非显而易见的部分是计算其之前映射到1桶的键的数量。这个操作可以通过使用专属扫描来完成，我们稍后会详细说明。

对于映射到1【译注：原文为0】桶的键，其目标索引可以通过以下方式找到：

```
destination of a one = # zeros in total + # ones before
                    = (# keys in total - # ones in total) + # ones before
                    = input size - # ones in total + # ones before
```

所有映射到0桶的键必须位于输出数组中映射到1桶的键之前。因此，映射到1桶的键（即1的目标）的目标索引等于映射到0桶的键的总数（即# zeros in total）加上该键之前的所有映射到1桶的键的数量（即# ones before）。由于所有键要么映射到0桶，要么映射到1桶，映射到0桶的键的总数等于输入列表中的总键数（即# keys in total）减去映射到1桶的键的总数（即# ones in total）。输入列表中的键总数就是输入的大小，这是显而易见的。因此，找到映射到1桶的键的目标索引的唯一复杂部分是计算之前有多少键映射到1桶，这与0桶的情况需要的信息相同。同样，这个操作可以通过使用exclusive scan来完成，如下文所述。映射到1桶的键的总数可以作为exclusive scan操作的副产品得到。

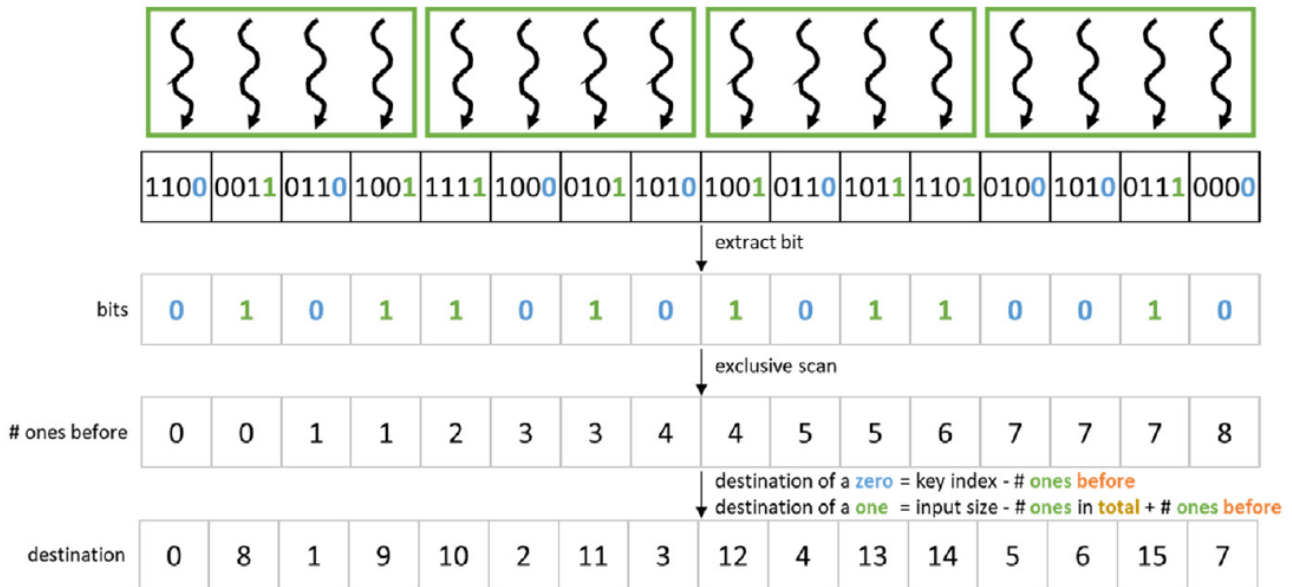


图13.3 找到每个输入键的目标位置。

图13.3展示了每个线程在图13.2的示例中为找到其键的目标索引所执行的操作。执行这些操作的对应内核代码如图13.4所示。首先，每个线程确定它负责的键的索引（第03行），执行边界检查（第04行），并从输入列表中加载键（第06行）。接下来，每个线程从键中提取当前迭代的位，以确定它是0还是1（第07行）。

```

01 __global__ void radix_sort_iter(unsigned int* input, unsigned int* output,
02                                unsigned int* bits, unsigned int N, unsigned int iter) {
03     unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
04     unsigned int key, bit;
05     if(i < N) {
06         key = input[i];
07         bit = (key >> iter) & 1;
08         bits[i] = bit;
09     }
10     exclusiveScan(bits, N);
11     if(i < N) {
12         unsigned int numOnesBefore = bits[i];
13         unsigned int numOnesTotal = bits[N];
14         unsigned int dst = (bit == 0)?(i - numOnesBefore)
15                             : (N - numOnesTotal - numOnesBefore);
16         output[dst] = key;
17     }
18 }

```

图13.4 基数排序迭代核代码。

在这里，迭代编号iter告诉我们感兴趣的位的位置。通过将键向右移此数值，我们将该位移动到最右边的位置。通过对移位后的键和1进行按位与操作（&），我们将移位键中的所有位清零，除了最右边的位。因此，bit的值将是我們感兴趣的那个位的值。在图13.3的示例中，由于示例是针对迭代0，提取的是最不重要位（LSB），如bits行所示。

一旦每个线程从键中提取了感兴趣的位，它就将该位存储到内存中（第08行），并且线程协作执行该位的exclusive scan（第10行）。我们在第11章“前缀和（扫描）”中讨论了如何执行exclusive scan。对exclusive scan的调用是在边界检查之外执行的，因为线程在执行过程中可能需要执行障碍同步，因此我们需要确保所有线程都是活动的。为了跨整个网格同步所有线程，我们假设可以使用与第11章“前缀和（扫描）”中单遍扫描所用的复杂技术类似的技术。或者，我们可以终止内核，从主机调用另一个内核来执行扫描，然后再调用第三个内核来执行扫描之后的操作。在这种情况下，每次迭代将需要三个网格启动而不是一个。

exclusive scan操作产生的数组在每个位置包含该位置之前的位的总和。由于这些位要么为0要么为1，该位置之前的位的总和等于该位置之前1的数量（即映射到1桶的键的数量）。在图13.3的示例中，exclusive scan的结果如# ones before行所示。每个线程访问该数组以获得其位置之前1的数量（第12行）以及输入列表中1的总数（第13行）。然后每个线程可以使用我们之前推导的表达式来确定其键的目标位置（第14-15行）。确定目标索引后，线程可以继续将其负责的键存储在输出列表中的相应位置（第16行）。在图13.3的示例中，目标索引如destination行所示。读者可以参考图13.2来验证所获得的值确实是每个元素的正确目标索引。

13.4 优化内存合并

我们刚刚描述的方法在并行化基数排序迭代方面是有效的。然而，这种方法的一个主要低效之处在于，输出列表的写入模式无法充分合并。考虑图13.2中每个线程将其键写入输出列表的方式。在第一个线程块中，第一个线程写入0桶，第二个线程写入1桶，第三个线程写入0桶，第四个线程写入1桶。因此，连续索引值的线程并不一定写入连续的内存位置，导致合并效果不佳，每个warp需要发出多次内存请求。

回顾第6章《性能考虑》，有多种方法可以实现内核中的更好内存合并：(1)重新安排线程，(2)重新安排线程访问的数据，或(3)在共享内存中执行无法合并的访问，并在共享内存和全局内存之间以合并的方式传输数据。为了在本章中优化合并，我们将使用第三种方法。我们不会让所有线程以无法合并的方式将其键写入全局内存桶，而是让每个线程块在共享内存中维护自己的本地桶。也就是说，我们将不再执行图13.4所示的全局排序。而是，每个块内的线程首先在共享内存中执行块级本地排序，将映射到0桶和映射到1桶的键分开。之后，将桶从共享内存以合并的方式写入全局内存。

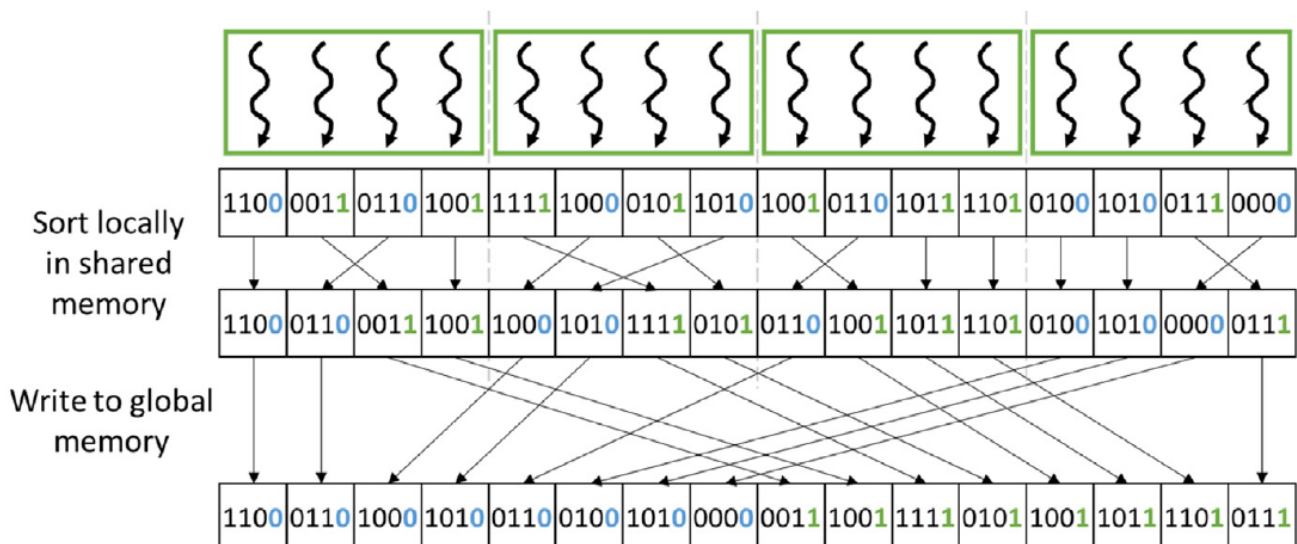


图13.5 通过在共享内存中进行本地排序，再排序到全局内存中，优化内存合并。

图13.5显示了如何为图13.2中的示例增强内存合并。在这个示例中，每个线程块首先对它拥有的键执行本地基数排序，并将输出列表存储到共享内存中。本地排序可以像以前的全局排序那样完成，只需每个线程块执行本地排他扫描，而不需要全局扫描。在本地排序之后，每个线程块以更合并的方式将其本地桶写入全局桶。例如，在图13.5中，考虑第一个线程块如何将其桶写入全局内存。当写入0桶时，前两个线程都写入全局内存的相

邻位置，而当写入1桶时，最后两个线程也写入全局内存的相邻位置。因此，大多数写入全局内存的操作将是合并的。

这种优化的主要挑战在于每个线程块需要识别其本地桶在相应全局桶中的起始位置。线程块的本地桶起始位置取决于其他线程块中本地桶的大小。特别是，线程块的本地0桶位置在所有前面线程块的本地0桶之后。另一方面，线程块的本地1桶位置在所有线程块的本地0桶和前面线程块的所有本地1桶之后。这些位置可以通过对线程块的本地桶大小进行排他扫描来获得。

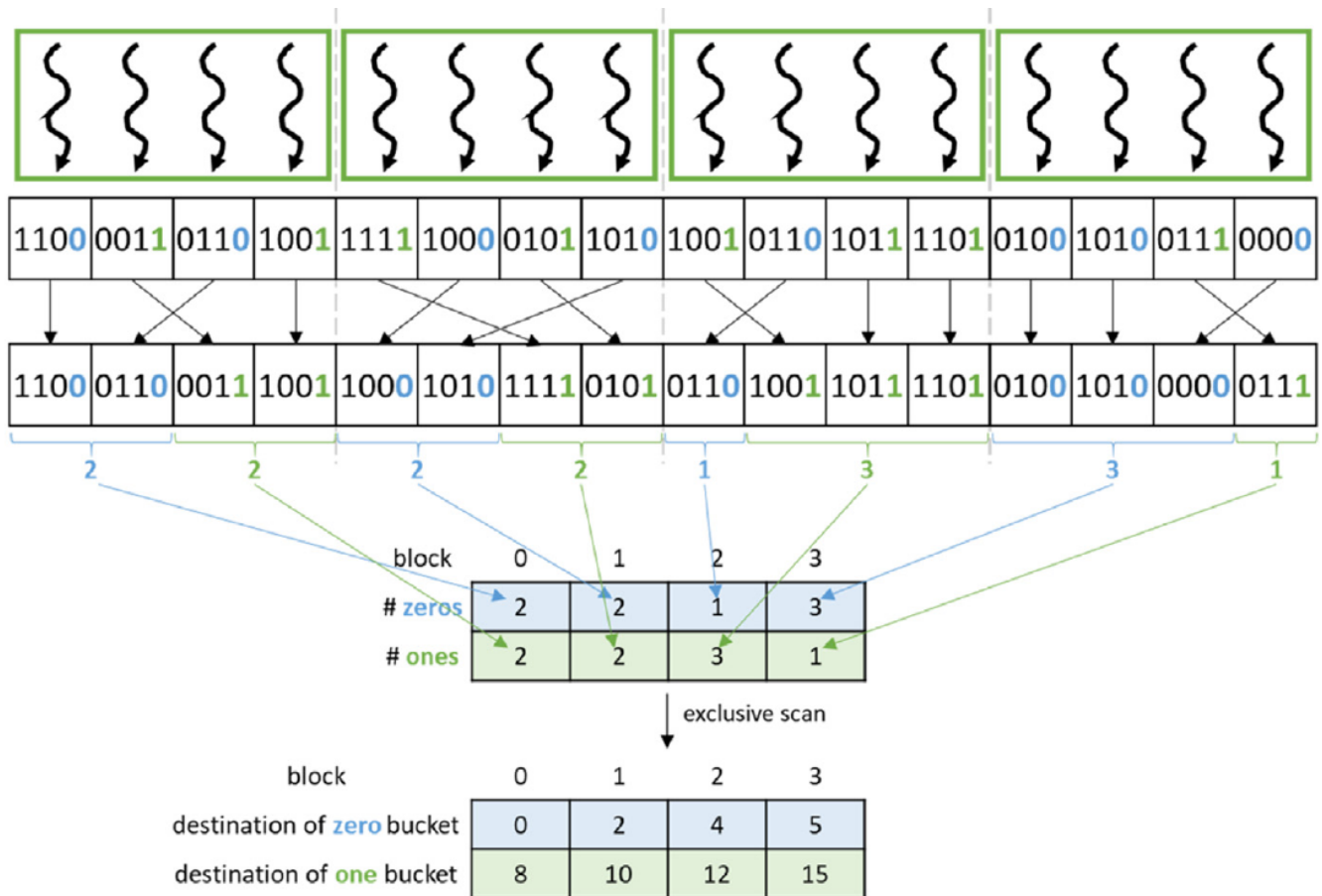


图13.6 确定每个线程块的本地桶在全局内存中的目标位置。

图13.6展示了如何使用排他扫描来找到每个线程块本地桶的位置。在完成本地基数排序后，每个线程块识别其本地桶中的键数量。接下来，每个线程块将这些值存储在一个表中，如图13.6所示。该表按行主顺序存储，即将所有线程块的本地0桶大小连续放置，随后是本地1桶的大小。表构建完成后，对线性化表执行排他扫描。生成的表包含每个线程块本地桶的起始位置，这些就是我们要找的值。

一旦线程块识别出其本地桶在全局内存中的起始位置，块中的线程就可以将其键从本地桶存储到全局桶中。为此，每个线程需要跟踪0桶与1桶中的键数量。在写入阶段，每个块中的线程将根据其线程索引值写入任意一个桶中的键。例如，对于图13.6中的块2，线程0写入0桶中的单个键，而线程1到3写入1桶中的三个键。相比之下，对于图13.6中的块3，线程0到2写入0桶中的三个键，而线程3写入1桶中的一个键。因此，每个线程需要测试其是否负责写入本地0桶或1桶中的键。每个块跟踪其两个本地桶中键的数量，以便线程确定其threadIdx值所在的位置并参与写入0桶键或1桶键。我们将该优化的实现留作读者练习。

13.5 基数值的选择

到目前为止，我们已经看到如何通过使用1位基数来并行化基数排序。对于示例中的4位键，需要进行四次迭代（每次处理一位）才能完全排序。一般来说，对于N位键，需要N次迭代才能完全排序。为了减少所需的迭代次数，可以使用更大的基数值。

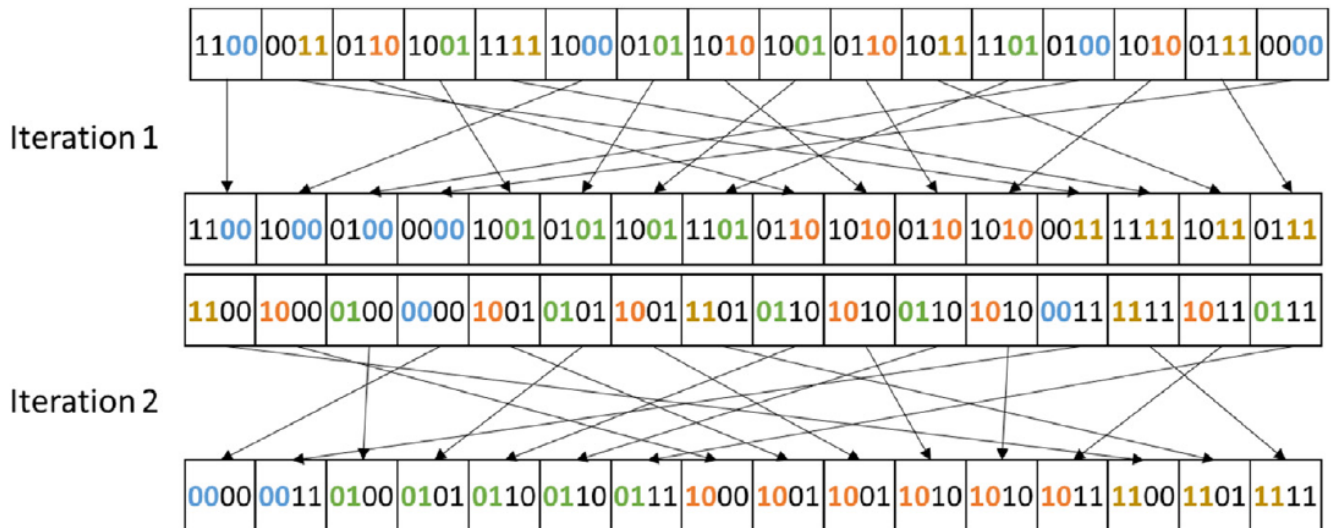


图13.7 2位基数的基数排序示例。

图13.7展示了如何使用2位基数执行基数排序的示例。每次迭代使用两位将键分配到桶中。因此，4位键只需两次迭代即可完全排序。在第一次迭代中，考虑低两位。键根据低两位的00、01、10和11分配到四个桶中。在第二次迭代中，考虑高两位。然后根据高两位将键分配到四个桶中。与1位示例类似，每个桶内键的顺序保留自上次迭代。保留每个桶内键的顺序确保在第二次迭代后，所有四位都被完全排序。

与1位示例类似，每次迭代可以通过为输入列表中的每个键分配一个线程来并行化，以找到键的目标索引并将其存储在输出列表中。为了优化内存合并，每个线程块可以在共享内存中本地排序其键，然后以合并的方式将本地桶写入全局内存。图13.8展示了如何并行化基数排序迭代并使用共享内存优化内存合并的示例。

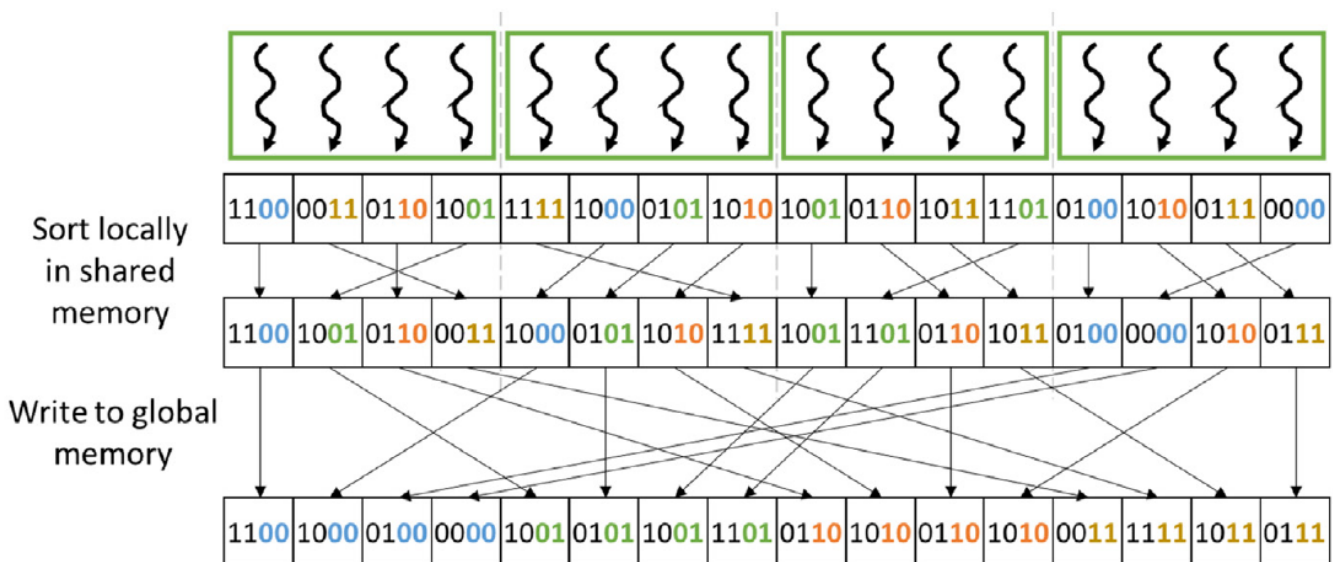


图13.8 并行化2位基数的基数排序迭代，并使用共享内存优化内存合并。

1位示例和2位示例之间的关键区别在于如何将键分为四个桶而不是两个。对于每个线程块内部的本地排序，通过应用两次连续的1位基数排序迭代来执行2位基数排序。每个1位迭代都需要其自己的排他扫描操作。然而，这些操作是线程块本地的，因此在两次1位迭代之间不需要跨线程块协调。一般来说，对于 r 位基数，需要进行 r 次本地1位迭代来将键排序到 2^r 个本地桶中。

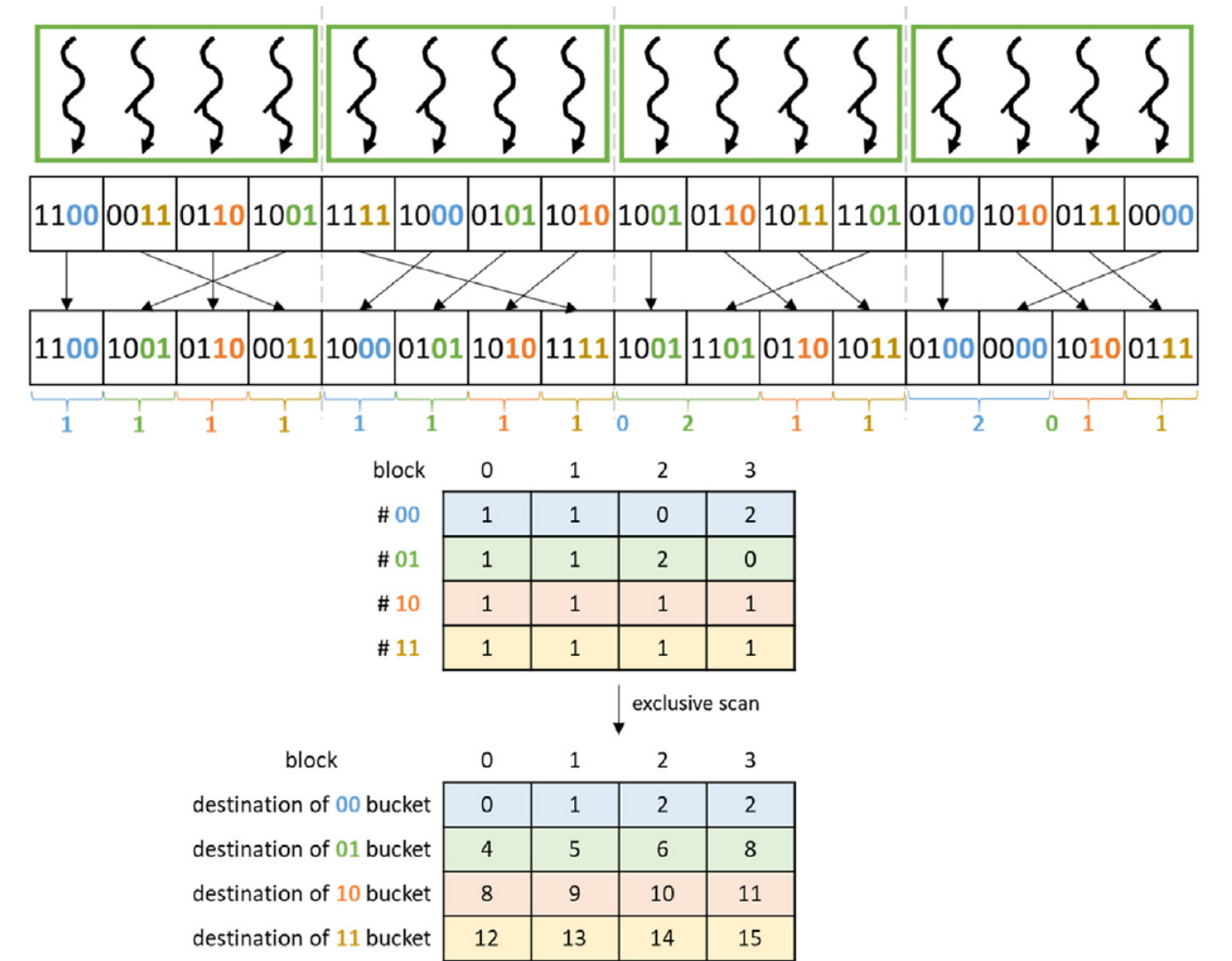


图13.9 确定每个块的本地桶在2位基数中的目标位置。

本地排序完成后，每个线程块必须找到其每个本地桶在全局输出列表中的位置。图13.9展示了如何找到2位基数示例中每个本地桶的目标位置。该过程与图13.6中的1位示例相似。每个线程块将每个本地桶中的键数量存储在一个表中，然后扫描该表以获得每个本地桶的全局位置。与1位基数示例的主要区别在于，每个线程块有四个本地桶而不是两个，因此排他扫描操作是在一个有四行而不是两行的表上执行的。一般来说，对于r位基数，排他扫描操作在一个有 2^r 行的表上执行。

我们已经看到，使用更大基数的优点是减少了完全排序键所需的迭代次数。较少的迭代意味着较少的网格启动、全局内存访问和全局排他扫描操作。然而，使用更大基数也有缺点。第一个缺点是每个线程块有更多的本地桶，每个桶中的键更少。因此，每个线程块需要写入更多的全局内存桶部分，而每个部分写入的数据更少。因此，随着基数变大，内存合并的机会减少。第二个缺点是应用全局排他扫描的表随着基数变大而变大。因此，全局排他扫描的开销随着基数增大而增加。因此，基数不能任意大。基数值的选择必须在迭代次数、内存合并行为和全局排他扫描的开销之间取得平衡。我们将多位基数基数排序的实现留作读者练习。

13.6 线程粗化以改善合并

在许多线程块上并行化基数排序的代价是写入全局内存时合并效果差。每个线程块都有自己的本地桶，它将这些桶写入全局内存。拥有更多的线程块意味着每个线程块的键更少，这意味着本地桶会更小，在写入全局内存时合并的机会更少。如果这些线程块要并行执行，那么合并效果差的代价可能是值得支付的。然而，如果这些线程块被硬件序列化，那么这个代价就没有必要支付。

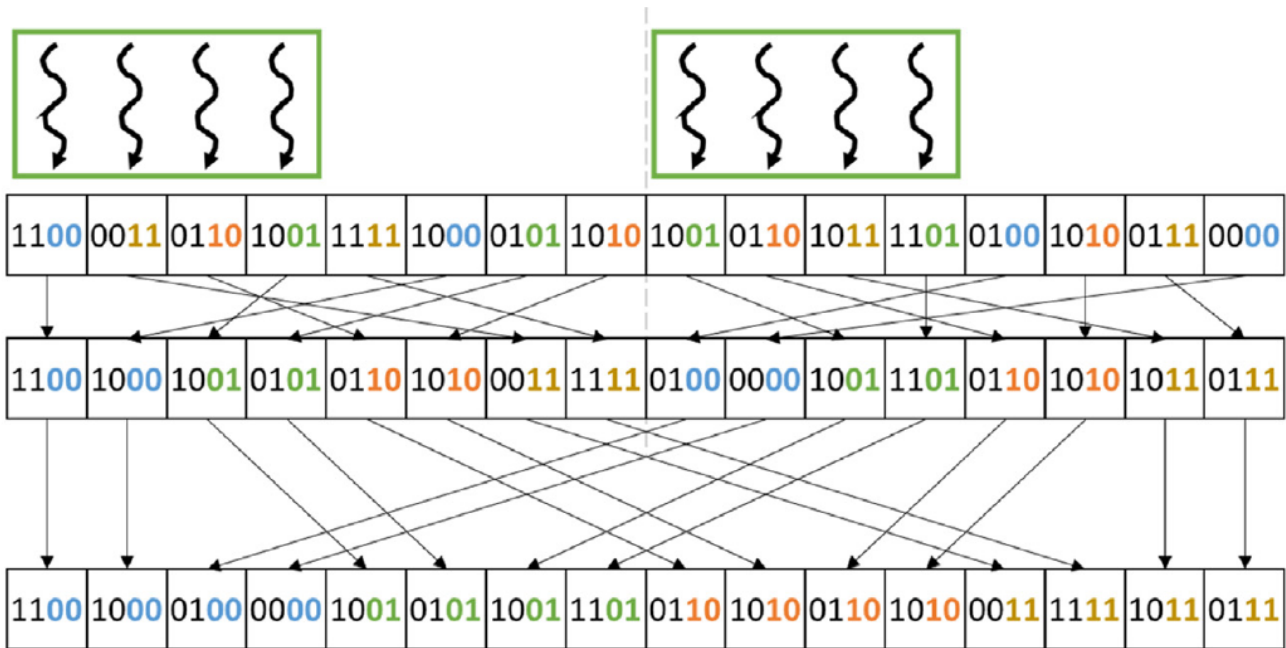


图13.10 使用线程粗化来改善内存合并的2位基数基数排序。

为了解决这个问题，可以应用线程粗化，其中每个线程被分配到输入列表中的多个键，而不仅仅是一个。图13.10展示了如何将线程粗化应用于2位基数例子的基数排序迭代。在这种情况下，每个线程块负责的键比图13.8中的例子更多。因此，每个线程块的本地桶更大，提供了更多的合并机会。当我们比较图13.8和图13.10时，很明显在图13.10中，连续线程写入连续内存位置的可能性更大。

并行化基数排序在许多线程块上的另一个代价是执行全局独占扫描以确定每个线程块的本地桶目的地的开销。回想一下图13.9，执行独占扫描的表的大小与桶的数量和块的数量成正比。通过应用线程粗化，减少了块的数量，从而减少了表的大小和独占扫描操作的开销。我们将线程粗化应用于基数排序的实现留给读者作为练习。

13.7 并行归并排序

基数排序适用于按字典顺序排序键。然而，如果键需要根据由复杂比较运算符定义的复杂顺序进行排序，则基数排序不适用，需要使用基于比较的排序算法。此外，通过简单地更改比较运算符，可以更容易地将基于比较的排序算法的实现适应于不同类型的键。相比之下，将非比较型排序算法（如基数排序）的实现适应于不同类型的键可能涉及创建实现的不同版本。这些考虑可能使基于比较的排序在某些情况下更受欢迎，尽管它们的复杂度更高。

一种适合并行化的基于比较的排序是归并排序。归并排序通过将输入列表划分为多个段，对每个段进行排序（使用归并排序或其他排序算法），然后对排序后的段进行有序合并来工作。

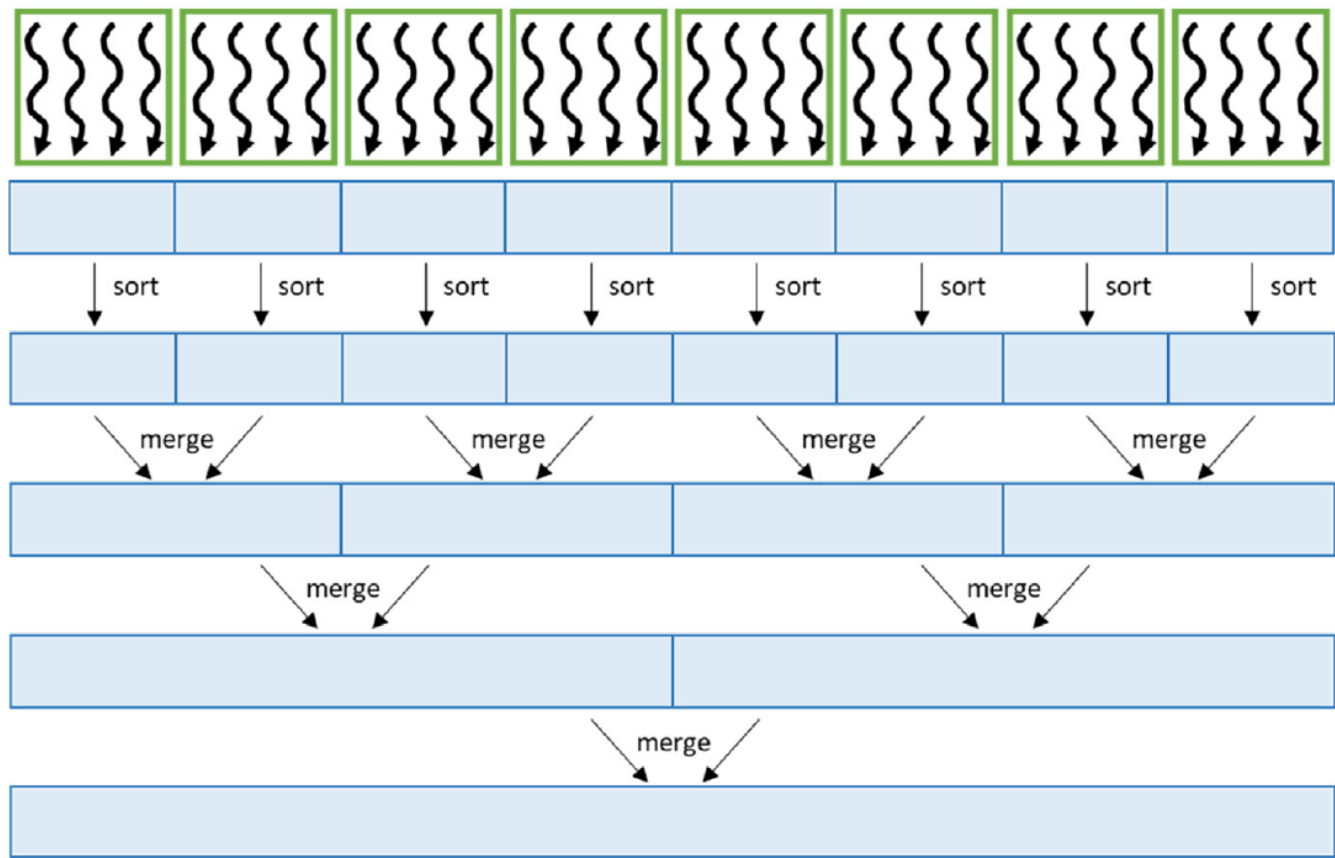


图13.11 并行化归并排序。

图13.11展示了如何并行化归并排序的一个示例。最初，输入列表被划分为许多段，每个段独立地使用某种高效的排序算法进行排序。之后，每对段被合并为一个单独的段。此过程一直重复，直到所有键成为同一个段的一部分。

在每个阶段，可以通过并行执行不同的合并操作以及在合并操作内部并行化来并行化计算。在早期阶段，可以并行执行更多的独立合并操作。在后期阶段，独立合并操作较少，但每个合并操作合并的键更多，从而在合并操作内部暴露出更多的并行性。例如，在图13.11中，第一阶段合并包括四个独立的合并操作。因此，我们的八个线程块的网格可能会分配两个线程块并行处理每个合并操作。在下一个阶段，只有两个合并操作，但每个操作合并的键数是之前的两倍。因此，我们的八个线程块的网格可能会分配四个线程块并行处理每个合并操作。我们在第12章中看到了如何并行化合并操作。我们将基于并行合并的归并排序的实现留给读者作为练习。

13.8 其他并行排序方法

上述算法只是并行排序数据的许多可能方式中的两种。在本节中，我们将简要概述一些可能对读者感兴趣的其它方法。

最简单的并行排序方法之一是奇偶位变换排序。它首先并行比较每对奇偶索引的键，即从第一个偶数索引开始的索引 k 和 $k+1$ 处的键。如果位置 $k+1$ 上的键小于位置 k 上的键，则交换它们的位置。然后，对从第一个奇数索引开始的每对偶奇索引的键重复此步骤。这些交替阶段一直重复，直到两个阶段都完成且没有键需要交换。奇偶位变换排序与顺序冒泡排序算法非常相似，并且像冒泡排序一样，它在大型序列上效率低下，因为它可能对 N 个元素的序列执行 $O(N^2)$ 的工作。

变换排序使用固定的比较模式，并在元素顺序错误时交换元素。由于每步比较的键对不重叠，因此它可以很容易地并行化。有一整类排序方法使用固定模式的比较来排序序列，通常是并行的。这些方法通常被称为排序网络，最著名的并行排序网络是Batcher的双调排序和奇偶合并排序 (Batcher, 1968年)。Batcher算法操作固定长度的序列，并且比奇偶位变换排序更有效，对于 N 个元素的序列只需要 $O(N\log N)$ 的比较。尽管这些算法的

成本在渐进意义上比归并排序等方法的 $O(N\log N)$ 成本更差，在实践中，由于它们的简单性，它们通常是小程序上最有效的方法。

大多数不使用排序网络典型固定比较集的基于比较的并行排序可以分为两个广泛的类别。第一类将未排序的输入划分为瓦片，对每个瓦片进行排序，然后在组合这些瓦片以形成输出时完成大部分工作。我们在本章中描述的归并排序就是这样的算法的一个例子；大部分工作在合并排序瓦片的归并树中完成。第二类主要关注对未排序序列的划分，以便合并分区相对简单。样本排序算法（Frazer和McKellar，1970年）是这一类别的典型例子。样本排序首先从输入中选择 $p-1$ 个键（例如，随机选择），对它们进行排序，然后使用它们将输入划分为 p 个桶，使得桶 k 中的所有键都大于任何桶 j 中的所有键，并且小于任何桶 j' 中的所有键，其中 $j < k$ 。这一步类似于快速排序执行的双向划分的 p 路概括。通过这种方式划分数据后，每个桶可以独立排序，通过简单地按顺序连接桶来形成排序后的输出。样本排序算法通常是处理必须跨多个物理内存分布的极大序列最有效的选择，包括在单个节点上的多个GPU的内存中。

在实践中，通常对键进行过采样，因为适度的过采样将以高概率产生平衡的分区（Blelloch等人，1991年）。

正如归并排序和样本排序分别代表了基于比较的排序的自底向上和自顶向下策略，基数排序算法也可以设计为遵循自底向上或自顶向下策略。我们在本章中描述的基数排序更准确地说是LSB，或者更一般地说，是最不显著位（LSD）基数排序。算法的连续步骤从键的最不显著位开始，向最显著位（MSD）工作。MSD基数排序采用相反的策略。它首先使用MSD将输入划分为对应于该位可能值的桶。然后，每个桶中独立应用相同的划分，使用下一个MSD。到达LSD时，整个序列将已排序。像样本排序一样，MSD基数排序通常是非常大的序列的更好选择。虽然LSD基数排序在每一步都需要全局数据洗牌，但MSD基数排序的每一步都操作在数据的越来越局部化的区域。

13.9 总结

在本章中，我们看到了如何在GPU上并行地对键（及其相关值）进行排序。在本章的大部分内容中，我们专注于基数排序，它通过将键分配到不同的桶中来进行排序。分配过程针对键中的每个数字重复执行，同时保持前一个数字迭代的顺序，以确保最终根据所有数字对键进行排序。每次迭代都通过为输入列表中的每个键分配一个线程，并让该线程找到键在输出列表中的目的地来并行化，这涉及到与其他线程协作执行独占扫描操作。

优化基数排序的一个关键挑战是在将键写入输出列表时实现合并的内存访问。一个重要的优化是让每个线程块对共享内存中的本地桶执行局部排序，然后以合并的方式将每个本地桶写入全局内存。另一种优化是增加基数的大小以减少所需的迭代次数，从而减少启动的网格数量。然而，基数大小不应增加过多，因为这会导致合并效果变差，并增加全局独占扫描操作的开销。最后，应用线程粗化在改善内存合并以及减少全局独占扫描的开销方面也是有效的。

基数排序的优势在于其计算复杂度低于 $O(N\log(N))$ 。然而，基数排序只适用于有限类型的键，如整数。因此，我们还研究了适用于通用类型键的基于比较的排序的并行化。一类适合并行化的基于比较的排序算法是归并排序。归并排序可以通过并行执行不同输入段的独立合并操作以及在每个合并操作内部进行并行化来并行化，正如我们在第12章“合并”中看到的。

在GPU上实现和优化并行排序算法的过程是复杂的，普通用户更有可能使用GPU的并行排序库，如Thrust（Bell和Hoerberock，2012），而不是从头开始实现自己的排序内核。尽管如此，并行排序仍然是研究优化并行模式所需权衡的一个有趣案例研究。